



Republic of Tunisia
Ministry of Higher Education and Scientific Research
University of Tunis El Manar
National Engineering School of Tunis



Engineer Diploma Thesis

Presented in order to graduate as

SOFTWARE AND DATA ENGINEER

Design and implementation of an automated Big Data solution for processing and analyzing charging system data

Defended by:
Firas Khemir

On: 28/09/2024

Hosting Company:



Chairman:	Mr. Khaled Benlamine
Reviewer:	Mr. Wady Naanaa
Company Supervisor:	Mr. Bilel Boussaa
Academic Supervisor:	Ms. Wafa Mefteh

Academic year:
2023-2024

Company Supervisor Appreciation

favorable opinion.

27 July 2024



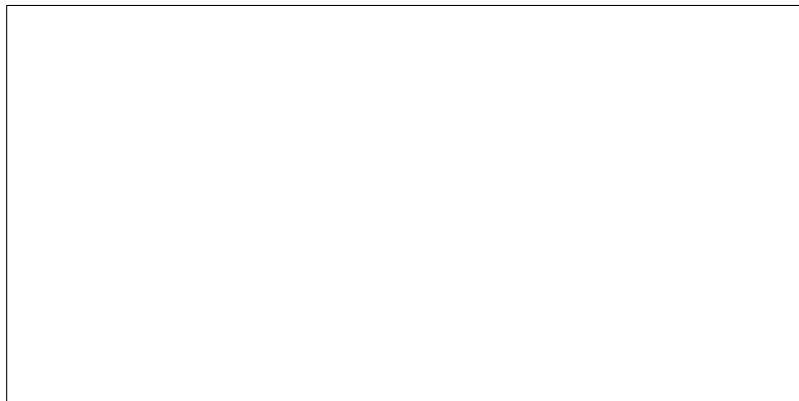
Lilil Boukha
Ingenieur chargé IN/VAS.

Academic Supervisor Appreciation

favorable opinion.

27 July 2024

Academic Supervisor Wafa Mefteh



Dedication

To my beloved family, whose unwavering support, encouragement, and love have been my guiding light throughout this journey.

To my parents, for their endless sacrifices and belief in my dreams, and to my siblings, for their constant motivation and companionship. I am forever grateful for their presence in my life.

To my friends, thank you for your kindness, patience, and understanding. Your encouragement has helped me persevere through the challenges and has made this journey much more enjoyable.

To all the teachers and mentors who have guided me over the years, your wisdom, dedication, and passion for education have shaped me into the person I am today. I am deeply indebted to you for your efforts and belief in my potential.

Last but not least, I would like to thank the jury for providing me with the opportunity to present my work and for accepting to evaluate it.

Acknowledgment

I would like to express my deepest appreciation and gratitude to all those who have supported and guided me throughout my internship project and the completion of this end-of-studies report. Their support and invaluable contributions have played a significant role in the successful completion of this endeavor.

First and foremost, I would like to express my gratefulness for being part of a brilliant team led by Mr Naoufel Fitouhi . Their dedication to our professional growth and development, coupled with their expertise and exceptional management skills, has been truly inspiring.

To Mr. Bilel Boussaa, for his unwavering guidance and mentorship throughout the duration of this project. Your expertise and patience have been invaluable to my growth and learning.

I equally feel thankful to the entire team at Orange Tunisie, for their continuous support, cooperation, and willingness to share their expertise. Working alongside such talented individuals has been a tremendous learning experience.

To Ms. Wafa Mefteh, my academic supervisor, for her exceptional support, guidance, and inspiration. Your insightful feedback have significantly contributed to the success of this project. Your commitment to my academic and personal development has been truly remarkable.

Abstract

This project was conducted during the end-of-studies internship at **Orange Tunisie**, focusing on building and implementing a big data solution for data reporting and exploitation in the Charging system Node of the Intelligent Network. The main challenge addressed is ingesting large amounts of coded data, decoding it, then providing it for further use.

Résumé

Ce projet a été mené pendant le stage de fin d'études chez **Orange Tunisie**, en se concentrant sur la construction et la mise en œuvre d'une solution big data pour le reporting et l'exploitation des données dans le nœud du système de charge du réseau intelligent. Le principal défi abordé est l'ingestion de grandes quantités de données codées, leur décodage, puis leur mise à disposition pour une utilisation ultérieure.

ملخص

تم تنفيذ هذا المشروع خلال فترة التدريب الداخلي لنهاية الدراسة في شركة أورانج تونس، مع التركيز على بناء وتنفيذ حل البيانات الضخمة للإبلاغ عن البيانات واستغلالها في عقدة نظام الشحن في الشبكة الذكية. ويتمثل التحدي الرئيسي الذي تمت معالجته في استيعاب كميات كبيرة من البيانات المشفرة وفك تشفيرها ثم توفيرها لمزيد من الاستخدام.

Contents

List of Figures	vi
General introduction	1
1 General Context	3
1.1 Host Company	3
1.1.1 Presentation of Orange Tunisia	3
1.1.2 Orange Tunisia Activities	4
1.2 Project Overview	5
1.3 Work methodology	5
1.4 Overview on ASN encoding	7
1.5 ASN.1 in telecommunications	8
2 Analysis and design	9
2.1 Ericsson Charging system	9
2.2 Data types and description	12
2.3 Requirements analysis	14
2.4 Solution Proposed and underlying architecture	17
3 Implementation	22
3.1 Work environment	22
3.2 Technological choices	24
3.3 Implementation steps	25
3.4 Software architecture	28
General Conclusion and Perspectives	30
Bibliography	32

List of Figures

1.1	Divisions of Orange Tunisia	4
1.2	Incremental Model Stages	6
2.1	Overview of the charging system	10
2.2	Solution location in the charging system.	11
2.3	Raw data format.	12
2.4	Data flow.	13
2.5	Decoded SDP message.	14
2.6	Solution Design.	19
2.7	Deployment Diagram.	20
2.8	Activity Diagram.	21
3.1	Software Architecture.	29

Acronyms

AIR account information and refill server.

ASN Abstract Syntax Notation.

CCN charging control node.

CDR call data record.

CRS charging and data reporting system.

CS charging system.

IN intelligent network.

KPI key performance indicator.

MINSAT Mobile Intelligent Network System Administration Tool.

OCC online charging control.

OCS online charging system.

SDP service data point.

General Introduction

Like all major industries, which have seen revolutionary changes brought about by big data, the telecoms industry is no different. Thanks to big data analytics, telecoms providers can make better decisions to improve their infrastructure, services and, consequently, revenues.

Big data and the telecoms sector essentially deal with the same commodity: information. By finding new ways of transmitting different types of information, operators can also identify and develop new sources of information that can help them make better business decisions.

In our case, the use of Big Data in telecoms industry is collecting and presenting charging provisioning and usage information, answering customer inquiries in near real-time. This big data solution, which enables business-critical reports in the customer's business organization. It can also be used by external system to retrieve the Call and Account History information.

In this context, we can place this work to develop a big data solution for treating and reporting the charging data of the intelligent network at the incumbent operator Orange Tunisia designed by the multinational ERICSSON. The solution is designed to help the operational team access KPIs, troubleshoot customer complaints in real time. It will be deployed on a separate Proxmox cluster and run continuously, collecting data from the system and providing it for further use in the form of a web interface.

The report is organized as follows: the first chapter provides an introduction to the host company and offers an overview of IN (Intelligent Network). The second chapter is dedicated to presenting the analysis and design of the solution. It covers a range of theoretical and technical aspects essential for understanding the solution. The third chapter delves into the implementation details. It begins by focusing on the develop-

ment environment and the technological choices made during the project. Following that, it scopes on a step-by-step description of the implementation of the features developed.

Chapter 1

General Context

Introduction

The first chapter focuses on presenting the host company, followed by a description of the project, its objectives, and some crucial terms that will be used throughout this document.

1.1 Host Company

The overall goal of this section is to introduce Orange Tunisia, the hosting firm, by outlining its activities and services.

1.1.1 Presentation of Orange Tunisia

Orange Tunisie is the second private telecoms operator to obtain a mobile telephony license in Tunisia, awarded in July 2009 for 257.3 million dinars. It is also the first to be awarded a license to operate a 3G network and fixed-line telephony, breaking the monopoly of Tunisie Télécom in the latter respect.

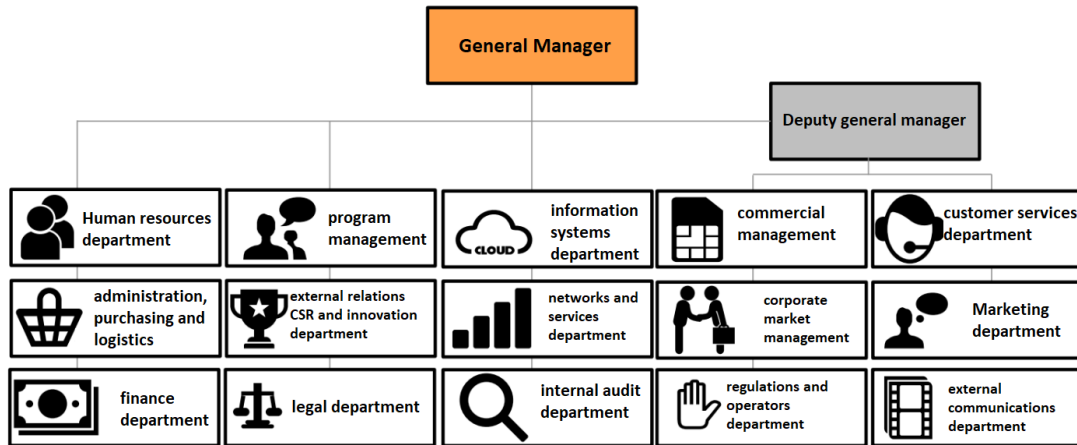


Figure 1.1: Divisions of Orange Tunisia

Orange tunisia has 12 divisions and 3 departments, illustrated in figure 1.1. which are under the supervision of a general manager and a deputy general manager.

1.1.2 Orange Tunisia Activities

Orange Tunisia is a company engaged in a wide range of activities, leveraging cutting-edge technologies and innovative solutions to deliver high-quality services. Its focus areas include software development, IT and network engineering, cybersecurity, system administration, cloud architecture, data science, and artificial intelligence. These technological advancements support various customer-centric initiatives, such as marketing product management, competitive intelligence, digital and social network engagement, external communication, as well as B2B and general sales and customer support.

In addition to its customer-facing efforts, Orange Tunisie also invests in vital support functions such as controlling and accounting, marketing analysis, performance data management, human resources, legal and regulatory compliance, real estate, and procurement. These functions play a crucial role in maintaining operational efficiency and ensuring the company's adherence to regulatory standards.

Furthermore, Orange Tunisie places strong emphasis on the support of its company-wide development initiatives. These include project and program management, process optimization, continuous improvement strategies, and team management, all aimed at fostering a culture of innovation and excellence within the organization.

1.2 Project Overview

Users can exchange calls and texts or consume data, for each of these events, a report is generated in a coded format to reduce its size. Naturally the report is needed so that we can charge the user for the transaction, or maybe troubleshoot a certain problem, for that we need a system capable of treating a large volume of data and keep up with huge flow of incoming new data, that's where our solution comes into play, we need to put in place an architecture capable of withstanding the large amounts of data while also keeping a reasonable response time and provide the tools necessary for troubleshooting

Project Background and Objectives:

Orange Tunisia already has a data reporting system, which is a computer system integrated into the intelligent network and connected to the network node. But this system has a number of drawbacks: it cannot process the data collected in real time, so it can't perform KPIs calculations in real time, transform data or detect anomalies. In addition, the application cannot evolve if other nodes are added to the charging system, and it is not fault-tolerant; if it fails, it has to be restarted, the restart time is more or less long, due to its deployment mode, so that each component of the application is deployed in a separate virtual machine. The solution must be scalable, fault-tolerant, highly available, process all kinds of data in real time, detect anomalies and make meaningful visualizations.

1.3 Work methodology

Work methodology is the systematic approach to organizing and managing the way people complete tasks. It emphasizes methodical planning and organized procedures that increase production and efficiency. For this project, we have chosen to employ the Incremental methodology, a software development strategy that emphasizes building large and complex systems by progressively adding new features. Illustrated in the figure 1.2 below, this approach does not require the entire system to be developed from the outset. Instead, it allows for continuous enhancements and iterative improvements over time. This methodology provides significant advantages, including flexibility, adaptability, and efficient development, making it well-suited for managing the complexities and evolving requirements of the system.

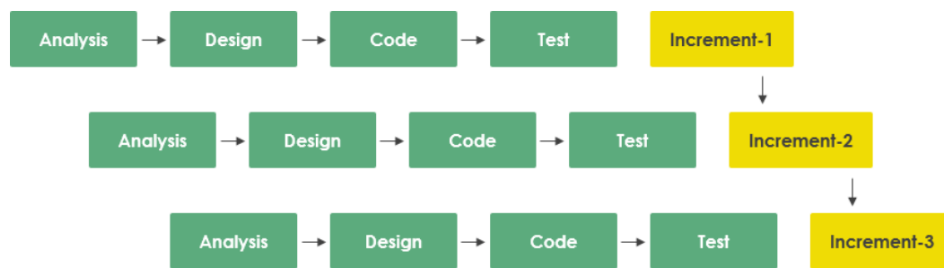


Figure 1.2: Incremental Model Stages

- **Steps of Incremental Model:**

- *Requirement Analysis:* In this initial phase, the primary focus is on gathering and analyzing project requirements, covering key aspects such as scope, objectives, and constraints. This phase also includes defining a clear project plan and identifying the essential resources needed for successful project completion. Once the documentation is prepared, it undergoes a thorough review and revision process before moving on to the next phase.
- *Design:* This phase marks the introduction and exploration of critical aspects such as system design and development methodologies. During this stage, architectural styles are introduced and analyzed, and the sequential steps of the development process are carefully examined.
- *Coding and implementation:* In this phase, the project advances by implementing code based on the specified product requirements. The primary objective is to develop clean, efficient code while strictly adhering to established coding standards. This stage bridges the gap between the design phase and the creation of a functional product, translating design specifications into a tangible, working solution through the practical coding process.
- *Test:* In the testing phase, the focus shifts to evaluating the compatibility and performance of the new solution. This involves rigorous testing of both newly introduced features and existing functionalities. During this phase, any necessary modifications and enhancements are identified and integrated to optimize the product's overall performance and capabilities, ensuring a high-quality final product.

1.4 Overview on ASN encoding

Abstract Syntax Notation One (ASN.1) is a standard interface description language (IDL) for defining data structures that can be serialized and deserialized in a cross-platform way. It is broadly used in telecommunications and computer networking, and especially in cryptography [3]

1.4.1 ASN.1 encoding explanation

ASN.1 specifies how data is represented and exchanged between different systems, ensuring compatibility regardless of platform or programming language. The data is encoded into a binary format, using encoding rules like BER (Basic Encoding Rules), DER (Distinguished Encoding Rules), and PER (Packed Encoding Rules), which make transmission more efficient.

ASN.1 structures are defined using a formal grammar, and the encoding process transforms the defined data structure into a binary format for transmission. On the receiving end, the binary data is decoded back into its original form using the corresponding ASN.1 definition.

Example

Consider a simple ASN.1 definition of a `Person`:

```
Person ::= SEQUENCE {
    name      UTF8String ,
    age       INTEGER
}
```

In this example, the `Person` type is defined as a sequence (a structured data type), consisting of a `name` (UTF-8 string) and an `age` (integer).

If we encode the data:

```
Person ::= {
    name = "John Doe" ,
    age = 30
}
```

Using BER encoding, this might be represented as:

```
30 0C 04 08 4A 6F 68 6E 20 44 6F 65 02 01 1E
```


Here, each byte (in hexadecimal) corresponds to a part of the encoded `Person` data: the tags and lengths of the fields, followed by the actual content (the name "John Doe" and the integer value 30).

ASN.1 is widely used in telecommunications, digital certificates (X.509), and various network protocols, where it ensures data integrity and seamless communication between different systems.

1.5 ASN.1 in telecommunications

Abstract Syntax Notation One (ASN.1) is a platform-independent language widely adopted in the telecommunications sector for specifying information structures. By defining data formats in a standardized way, ASN.1 facilitates reliable interoperability between diverse systems regardless of hardware or software differences. For example, it enables seamless interconnection between network equipment, mobile devices, and cellular towers built on dissimilar architectures. ASN.1 supports multiple encoding schemes, allowing for efficient use of transmission capacity even under network constraints. Basic Encoding Rules and Packed Encoding Rules optimization help maximize bandwidth utilization. Its rigorous syntactic structure ensures consistent interpretation of exchanged information. Misunderstandings are minimized, a critical property for protocols like GSM, LTE, and 5G where compatibility across global networks under varying conditions is paramount. ASN.1's precise definitions have made it integral to cross-platform communication in wireless telecom networks.

Conclusion

In the first chapter, we introduced the hosting company and gave a thorough overview of the project. We also delved into the specific problem that our solution aims to address and explained briefly one of the key concepts in the solution which is the encoding of transmitted data.

Chapter 2

Analysis and design

Introduction

This chapter begins by providing a comprehensive overview of the intelligent network at Orange Tunisie, it defines the charging system, then the relation between our solution in regards to the rest of the charging system, in addition to that, this chapter delves into the types of data we are dealing with and the challenges that they come with, finally the design of the solution proposed and different technologies used.

2.1 Ericsson Charging system

Ericsson's charging system is a robust and scalable solution designed to meet the complex billing and charging needs of telecom operators. It enables real-time charging for voice, data, and messaging services, providing operators with the flexibility to offer a wide range of pricing models, including prepaid, postpaid, and hybrid options. By integrating advanced analytics and machine learning capabilities, the system enhances customer experiences through personalized offerings and dynamic pricing strategies. Its modular architecture allows for seamless integration with existing systems, ensuring operators can adapt to evolving market demands and consumer behaviors efficiently. the chargin system general architecture is displayed in the figure 2.1

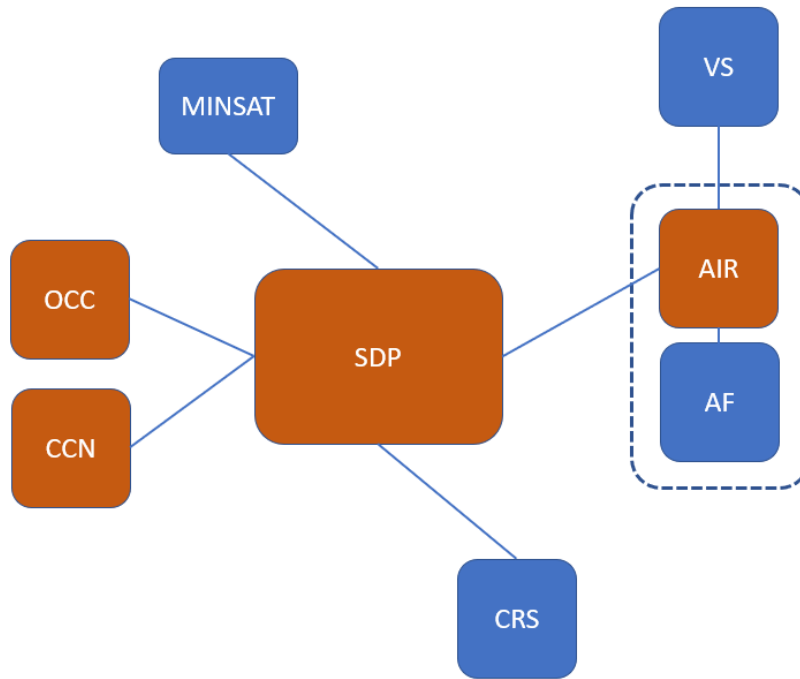


Figure 2.1: Overview of the charging system

Account Finder (AF): AF is a network element that provides SDP information for subscriber accounts in the system.

Account Information and Refill System (AIR): The AIR server handles messages from both traffical and administrative network elements, inside Charging System.

Service Data Point (SDP): The SDP network element contains the database with subscribers and account information.

Voucher Server (VS): VS is responsible for the administration and storage of vouchers.

Charging Control Node (CCN): CCN is responsible for protocol translation, session control, and coordination with other network elements. To perform this task, CCN supports a number of protocols, both towards the MSC and towards the different network elements.

Online Charging Control (OCC): OCC is responsible for protocol translation, session control, and coordination with other network elements. To perform this task, OCC supports a number of protocols, both towards the core network and towards the different network elements.

Charging data Reporting System (CRS): CRS is a system for correlating information from many different sources to reconcile provisioning and use information, and provides information for customer enquiries in near real time. CRS collects data from

various network sources, then filters, correlates, and transforms this information to provide business reports for customer care, financial, and audit departments.

Relation between our solution and the charging system

Our solution aims too attach itself to the highlighted nodes namely, SDP OCC CCN AIR, for the purpose of receiving data and decoding it, treating and then storing it locally for end user exploitation such as troubleshooting or resolving customer complaints. as illustrated int he figure 2.2

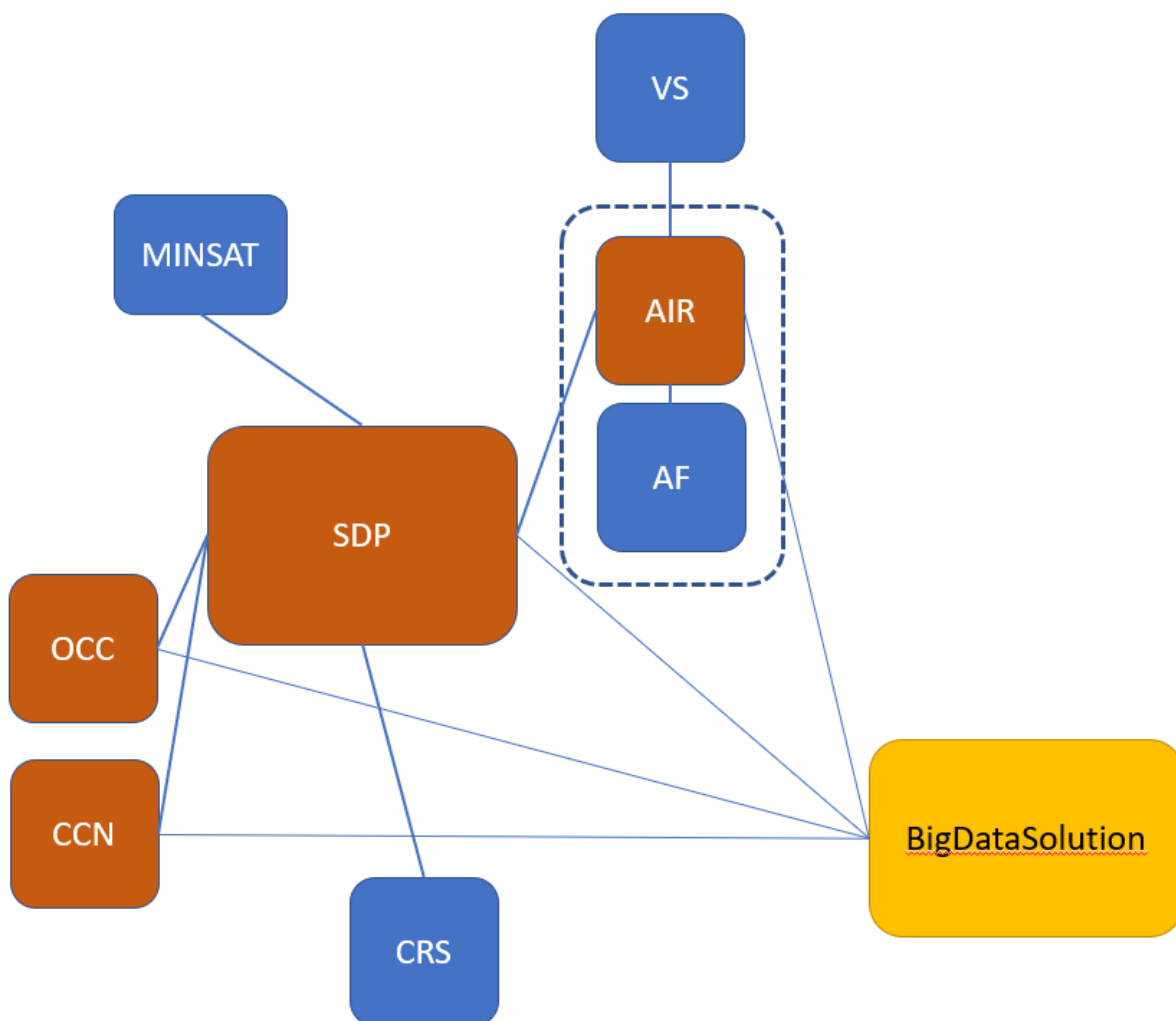


Figure 2.2: Solution location in the charging system.

2.2 Data types and description

When it comes to the data we are going to receive, it is coded using the ASN.1 encoding, resulting in much smaller files in size, ranging from less than a megabyte to 5 or 6 megabytes in some cases.

The raw data come in binary form such as the one illustrated in figure 2.3

Address	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F	Text
0000	A2	82	01	32	81	01	01	82	08	54	4E	32	53	44	50	34	...2....TN2SDP4
0010	61	83	13	32	32	37	31	31	33	37	36	39	35	38	35	34	a..2271137695854
0020	37	38	30	33	39	36	84	08	35	33	30	32	35	35	39	30	780396..53025590
0030	85	08	35	33	30	32	35	35	39	30	86	13	32	30	32	33	..53025590..2023
0040	30	37	30	33	32	32	33	30	31	30	2B	30	31	30	30	8B	0703223010.0100.
0050	01	16	8C	01	00	8F	03	45	58	54	90	0B	6F	6E	65	6D	EXT..onem
0060	6F	72	65	74	69	6D	65	91	0D	31	36	38	38	34	31	39	oretime..1688419
0070	38	31	30	38	35	38	92	13	32	30	32	33	30	37	30	33	810858..20230703
0080	32	32	33	30	31	30	2B	30	31	30	30	9E	08	31	30	30	223010.0100..100
0090	30	30	30	30	30	9F	1F	08	31	30	30	30	30	30	30	30	00000...10000000
00A0	9F	25	08	32	30	31	39	31	31	30	36	BF	49	25	30	23	...20191106.I.O.
00B0	80	02	01	7A	85	01	01	87	08	32	30	32	33	30	37	30	...z.....2023070
00C0	39	92	01	06	94	01	00	95	04	14	A0	00	00	96	04	14	9.....
00D0	A0	00	00	BF	4A	23	30	0F	80	01	10	81	01	00	82	01	J.O.....
00E0	01	83	01	01	84	01	01	30	10	80	02	01	7A	81	01	04	0....z...
00F0	82	01	00	83	01	04	84	01	02	9F	4E	03	41	49	52	BF	N.AIR.
0100	52	34	30	32	80	02	00	FA	81	01	05	83	08	32	30	32	R402.....202
0110	33	30	37	30	39	84	08	32	30	32	33	30	37	30	39	86	30709..20230709.
0120	08	32	30	32	33	30	32	30	33	82	08	32	30	32	33	30	20230703 20230

Figure 2.3: Raw data format.

As illustrated, the data is not comprehensible and can't be manipulated, therefore we have to implement ways of decoding this data to be able to use it. In addition to that the data flow is large and continuous and so we need a solution capable of handling this large workload and store it properly, this calls for the development of a big data solution capable of handling these challenges.

The solution should be able to ingest raw files (CDRs) decode them and then store the decoded files in a database capable of handling such a large amount of data, finally we should be able to index the data to be able to query it later, as illustrated in the figure 2.4 .

Ingest, Decode, Store, Index, Query



Figure 2.4: Data flow.

2.2.1 Data description

When it come to the data we are going to manipulate after decoding, it comes in several forms, each of the four nodes generates a unique type of files that's corresponding to it. for example SDP files are related to subscriber data, or other data required to process a call. For example, the subscriber's remaining prepaid credit may be stored in the SDP to be queried in real-time during the call. The figure 2.5 represents one decoded SDP message, extracted from a raw CDR.

When it comes to the AIR node, it generates files related to refilling subscriber accounts. It contains information needed for extending expiry dates and adding money to the subscriber accounts. The AIR system also provides information related to the subscriber accounts such as special offers and services.

The OCC files contain authorization requests, and ensures that charging policies are enforced correctly.

The CCN files contain information about short Message Service (SMS), In addition, CCN contains information for the Diameter Service Charging Application to support Content based services.

```

SDPCallDataRecord {
  accountAdjustment {
    adjustmentRecordType = refillCDR
    sdpID = TN2SDP4a
    cdrID = 2271137695854780396
    accountNumber = 53[REDACTED]
    subscriberNumber = 53[REDACTED]
    adjustmentTimeStamp = 20230703223010+0100
    serviceClassId = 22
    currencyType = 0
    originNodeType = EXT
    originNodeId = onemoretime
    origTransactionID = 1688419810858
    origTransactionTimeStamp = 20230703223010+0100
    accountFlagsBefore = 10000000
    accountFlagsAfter = 10000000
    activationDate = 20191106
    adjustmentDedicatedAccounts {
      AdjustmentDedicatedAccount {
        dedicatedAccountID = 378
        action = add
        accountExpiryDateAfter = 20230709
        accountUnitType = volume
        accountUnitBefore = 0
        accountUnitAfter = 346030080
        adjustmentUnit = 346030080
      }
    }
  }
}

```

Figure 2.5: Decoded SDP message.

2.3 Requirements analysis

2.3.1 Actor identification

The successful operation of this big data solution depends on the interactions of several key actors, each playing a critical role in utilizing and maintaining the system. These actors represent different user roles who engage with the system's data and infrastructure to achieve their specific goals. The following section outlines the roles and responsibilities of the primary actors.

IN Support Engineer

The IN Support Engineer is responsible for the ongoing maintenance, monitoring, and

troubleshooting of the system. This actor also uses the solution to solve customer complaints on a daily basis.

Business Intelligence (BI) Analyst

The BI Analyst focuses on deriving insights from the processed and stored data to support decision-making processes. This actor leverages the system's search and analytics capabilities to uncover patterns and trends in the data.

Fraud Detection Agent

The Fraud Detection Agent is tasked with using the system to identify and mitigate fraudulent activities. This actor relies on the system's ability to process, store, and search data in real-time to detect suspicious patterns or behaviors that may indicate fraud.

2.3.2 Functional and non-Functional requirements

Functional requirements

The functional requirements of this big data solution are centered around the core operations that the system must perform to meet its objectives. These include data ingestion, processing, storage, search, and retrieval functionalities, all essential for the system to operate as intended.

Real-time Data Ingestion: The system must be able to ingest data continuously and in real time from various sources. Apache Kafka is employed for this purpose, ensuring that the system can handle large volumes of streaming data without interruptions.

Data Decoding and Processing: Once the data is ingested, it must be decoded and processed using Apache Spark. This involves applying a custom decoding function to transform the raw binary data into a readable format. Additionally, the system must be capable of performing transformations, filtering, and aggregations on the data as it flows through the pipeline.

Data Storage: The decoded and transformed data must be stored in MongoDB. This storage system must support the creation of collections to store different types of data, such as raw files, decoded files, and metadata. The storage layer should also allow for easy retrieval of data in a structured format.

Search Functionality: The solution must provide a search feature that allows users

to query the stored data efficiently. Elasticsearch is used to index the data, enabling full-text search, filtering, and querying capabilities. The system should return accurate search results based on specified fields such as filename, content, accountNumber, and timestamp.

User Interface for Search: A web-based client, built using Flask, must offer a simple and intuitive interface where users can search the data. The client must allow users to submit queries and view results in real time. It should also support features like pagination, sorting, and filtering to navigate through large datasets.

Non Functional requirements

The non-functional requirements describe the overall system qualities that ensure its performance, scalability, security, and usability. These requirements are critical for the long-term reliability and maintainability of the solution.

Scalability: The system must be able to scale horizontally as the volume of data grows. This means Kafka should handle increased throughput as more data is ingested, Spark should efficiently process larger datasets in a distributed manner, and MongoDB and Elasticsearch should scale their storage and indexing capacities without a loss in performance. Docker's containerization also ensures that services can be scaled independently as needed.

Performance: The system must provide low-latency processing and quick response times for user queries. Data ingestion and processing should happen in near real time, ensuring minimal delays between data arrival and availability for search. The search functionality should return results within milliseconds, leveraging Elasticsearch's efficient indexing and query mechanisms.

Reliability and Fault Tolerance: The system must be resilient to failures and ensure continuous operation. Kafka's distributed nature and partitioning provide fault tolerance in data ingestion. Spark's distributed computing model ensures that data processing is not interrupted by individual node failures. MongoDB replication and Elasticsearch's clustering features provide redundancy, ensuring data availability even in case of hardware or network issues.

Security: Data must be handled securely throughout the entire pipeline. Secure communication channels between components (e.g., Kafka, Spark, MongoDB, Elasticsearch) should be enforced, potentially via SSL/TLS. Access controls should be implemented, ensuring that only authorized users can query or modify the data, particularly in the web client. Authentication and role-based access control can be applied to restrict sensitive data access.

Usability: The system must provide an intuitive and user-friendly interface for interacting with the data. The Flask web client should be simple and easy to navigate, requiring minimal technical knowledge to perform searches and retrieve results. Additionally, the system's API endpoints should be well-documented to facilitate integration with external tools and workflows.

Maintainability: The solution must be easy to maintain and extend over time. Using Docker for containerization simplifies the management of different services, ensuring that each component can be updated or replaced independently without affecting the overall system. The use of modular and well-documented code ensures that future developers can easily add new features or troubleshoot issues.

Availability and Uptime: The system must ensure high availability to support real-time operations. The use of distributed components like Kafka, Spark, MongoDB, and Elasticsearch, along with Docker orchestration, ensures that the system can remain operational with minimal downtime, even during maintenance or in the event of component failure.

2.4 Solution Proposed and underlying architecture

2.4.1 Introduction

This chapter delves into the crucial design phase of our project. We'll begin by outlining the overall design of our solution and then delve deeper into the specifics. To aid comprehension, we'll offer detailed descriptions of the project's architecture, complemented by diagrams that illustrate key concepts and structures.

2.4.2 General design

As mentioned above, our solution is going to attach itself to the charging system nodes and be able to ingest data automatically then decode it. for that we'll first need a strong data streaming solution, for that we'll be using **Apache Kafka**.

Once we retrieve the raw files we need to be able to decode them instantly for that we'll be using **PySpark** as it is good at Handling large datasets efficiently through distributed computing, allowing for operations like filtering, aggregating, and transforming data. In addition to being a strong stream processor: Analyzing real-time data streams, enabling applications such as monitoring and real-time analytics.

Once the data is decoded we should be able to store it in a resilient database capable of handling heterogeneous type of data and also capable of maintaining large datasets. for that we'll be using **MongoDB** as a document based database.

Once we have our decoded data stored, we should also be able to query it, for that we need a strong indexing tool capable of dealing with large datasets and semi-structured data. For this we'll be using **ElasticSearch** as it is a powerful, distributed search and analytics engine built on Apache Lucene. It is designed for handling large volumes of data in real time and is often used for full-text search, structured search, and analytics.

Last but not least we need a resilient front-end service to be able to exploit the decoded data. for that we'll be using **Flask** as lightweight web framework for Python that is designed for building web applications quickly and easily. It follows the WSGI (Web Server Gateway Interface) standard and is classified as a microframework because it provides the essential features needed to develop web applications without the overhead of more complex frameworks.

The solution's general design is further illustrated in the figure 2.6

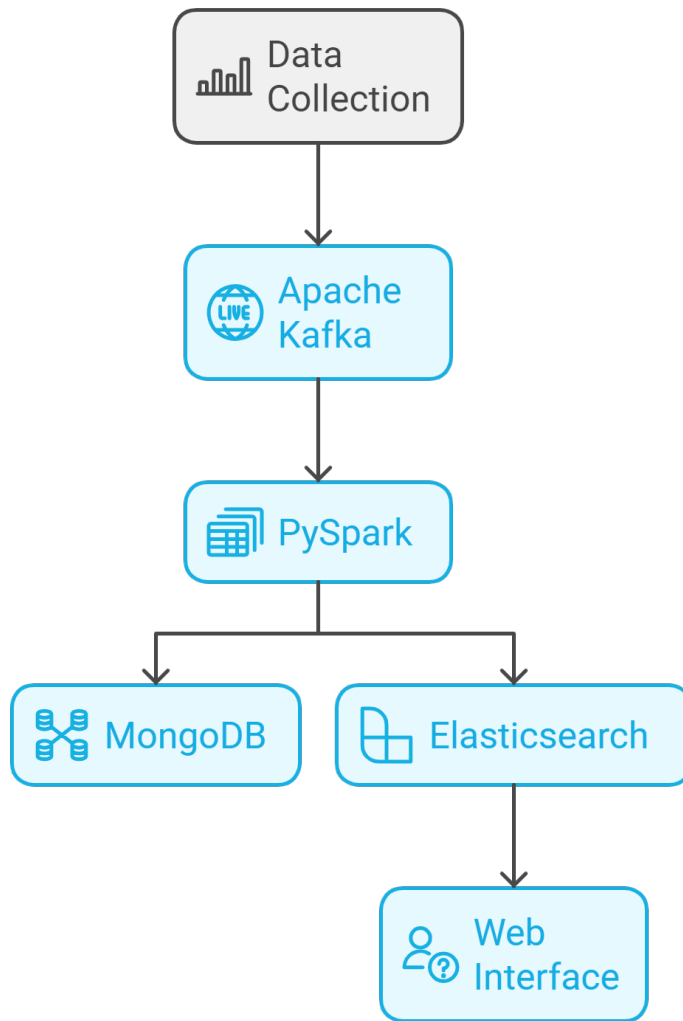


Figure 2.6: Solution Design.

2.4.3 Detailed design

The following presents a thorough and well-structured design of the solution, commencing with a detailed deployment diagram that outlines the system architecture, followed by an insightful activity diagram that captures the dynamic processes and workflows involved in the solution's operation.

2.4.3.1 Deployment Diagram

This architecture ensures that the system is scalable, reliable, and capable of handling multiple user requests efficiently.

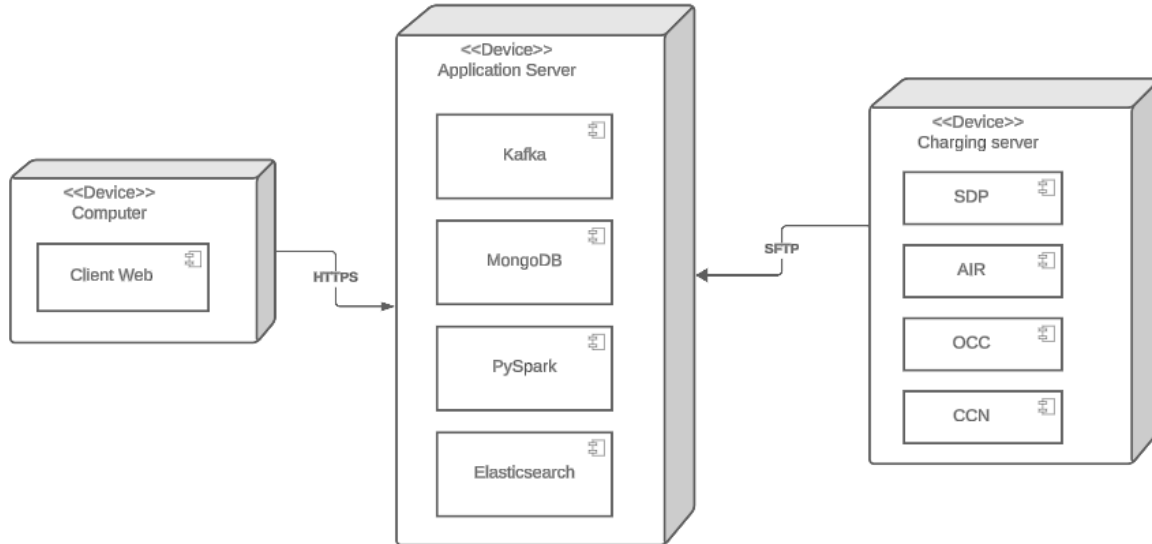


Figure 2.7: Deployment Diagram.

2.4.3.2 Activity Diagram.

An activity diagram can be defined as a visual representation, similar to a flowchart, that illustrates the sequence of activities performed by a system. It provides a clear description of the different actions and tasks performed within the system and demonstrates the flow of control between those activities [1] the figure 2.8 illustrates the activity diagram of our solution.

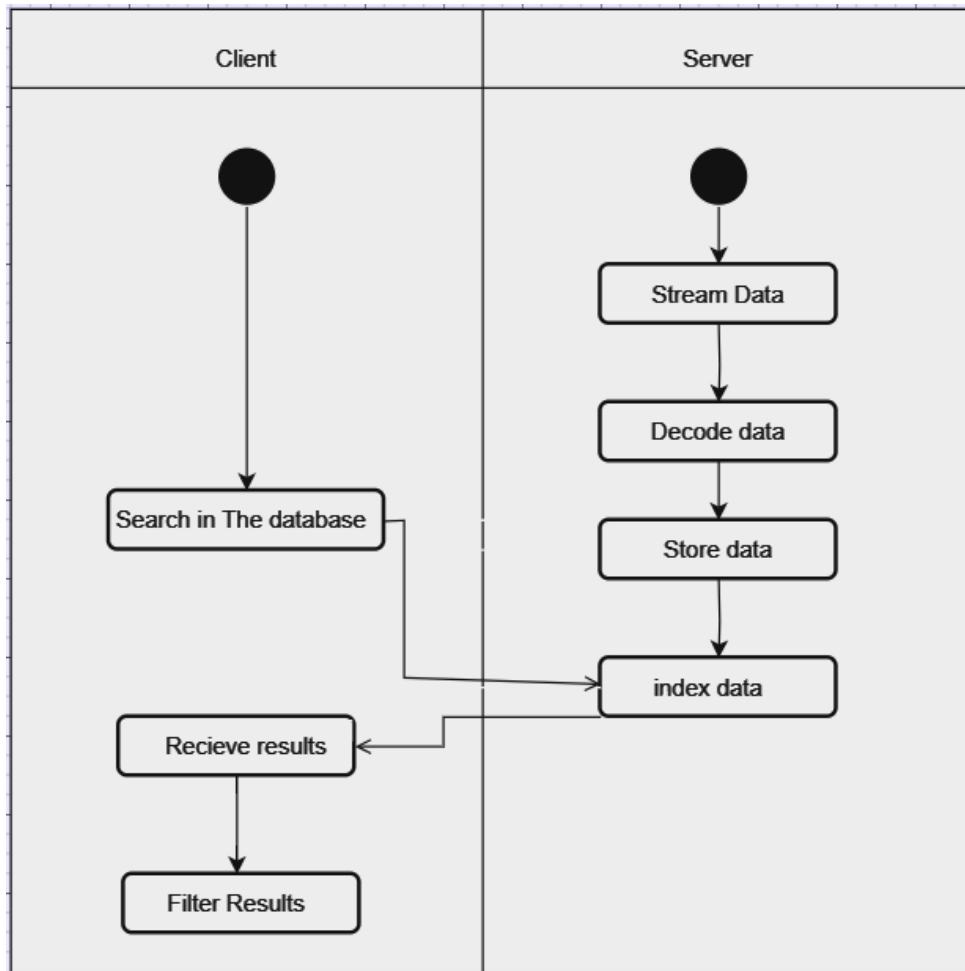


Figure 2.8: Activity Diagram.

Conclusion

In conclusion, this chapter offers a deep understanding of the intelligent network at Orange Tunisie, elucidating the critical role of the charging system and its integration with our proposed solution. By examining the various data types and the challenges they present, we establish a foundational context for the design of our solution. The insights gained here not only highlight the technological innovations employed but also underscore the importance of a well-structured approach in addressing the complexities inherent in modern telecommunications. As we move forward, these considerations will serve as essential guideposts for implementing effective and efficient solutions within the network.

Chapter 3

Implementation

Introduction

The successful realization of any big data solution hinges on the practical execution of its design and architecture. In this chapter, we will explore the detailed steps undertaken to implement the proposed big data architecture. The focus is on how various components—such as data ingestion, storage, processing, and visualization—are integrated into a cohesive system.

The solution utilizes a robust technology stack to process, store, and analyze large volumes of data in real-time. The implementation incorporates Apache Kafka for data streaming, Apache Spark for distributed data processing, MongoDB for document-oriented data storage, and Elasticsearch for powerful search capabilities.

This chapter provides a comprehensive walk-through of the deployment environment, detailing how the solution leverages containerization and streaming frameworks to handle real-time data processing and analytics efficiently.

3.1 Work environment

3.1.1 Hardware

During my internship, I worked with a robust and efficient hardware: a laptop equipped with a multi-core processor (Intel Core i7), 16 GB of RAM, and a high-speed SSD with Windows 10 as operating system.

3.1.2 Software

The implementation of the big data solution required a well-integrated software stack, ensuring the seamless orchestration of data ingestion, processing, storage, and visualization. Below is an overview of the key software components utilized in this project:

Docker

Docker was fundamental to the deployment of the various services used in this solution. By leveraging containerization, Docker ensured that each component—Kafka, Spark, MongoDB, Elasticsearch, and Kibana—could run in isolated, portable environments. Docker Compose was used to orchestrate the multi-container setup, facilitating easy management, scaling, and replication of the entire system. This containerized approach enhanced deployment speed and simplified dependency management across different components of the solution.

Apache Kafka

Apache Kafka was employed as the primary data streaming platform, handling real-time data ingestion. Kafka's distributed architecture enabled reliable message handling, making it the backbone of the solution's data pipeline. It allowed the continuous collection and distribution of data to the downstream processing engine, ensuring fault tolerance and scalability.[4]

Apache Spark

Apache Spark was integrated to perform distributed processing of data ingested through Kafka. Spark's ability to process large-scale data both in batch and streaming modes made it a critical component in real-time data transformation. Spark Structured Streaming was used to decode and process the data, leveraging its fault-tolerant and highly scalable processing capabilities.

MongoDB

MongoDB served as the primary data storage system for both raw and processed data. Its document-oriented database model was ideal for storing complex, unstructured data in JSON format. The flexibility of MongoDB made it suitable for handling a variety of data formats, while its scalability allowed for the seamless storage of large datasets.

Elasticsearch

Elasticsearch was implemented to provide powerful full-text search capabilities across

the stored data. The ability to index and query the data in near real-time allowed for efficient searching and retrieval of documents based on various criteria. Elasticsearch was a crucial addition to enhance the solution's search functionality, complementing MongoDB's storage capabilities.

Kibana

Kibana was integrated as the visualization tool for the data indexed in Elasticsearch. With its intuitive interface, Kibana allowed for the creation of dynamic dashboards, providing real-time insights into the data processed by Spark and indexed by Elasticsearch. This visualization tool was essential for monitoring trends, patterns, and overall system health.

MongoDB Compass

MongoDB Compass was utilized as the graphical user interface for MongoDB. It allowed for easy visualization, querying, and management of the data stored in MongoDB. By providing an intuitive way to inspect collections and documents, MongoDB Compass simplified database interactions, enhancing productivity during development and troubleshooting.

Visual Studio Code

VSCode served as the primary code editor for developing the solution. With its rich set of extensions, including support for Docker, Python, and Spark, VSCode offered an integrated environment for writing, debugging, and managing the codebase. Its seamless integration with Git also facilitated version control, making collaboration and code management more efficient throughout the development lifecycle.

3.2 Technological choices

The selection of the software stack for this big data solution was driven by a thorough comparison of available alternatives. Each tool was chosen based on its specific advantages in addressing the requirements of the system, including scalability, performance, ease of integration, and ecosystem support. Below, we outline the rationale behind these choices and highlight the benefits that led to the final selection.

Docker

For containerization, Docker was chosen over traditional virtual machines (VMs). While

VMs provide full isolation and can run different operating systems, Docker offers lightweight, containerized environments with minimal overhead. This is especially beneficial for a multi-component architecture like ours, where services like Kafka, Spark, and MongoDB need to communicate efficiently. Docker's faster startup times, lower resource consumption, and seamless orchestration using Docker Compose made it the superior choice compared to VMs, which would require more resources and complex networking configurations.

Apache Spark

For data processing, Apache Spark was chosen over alternatives like Apache Flink and Hadoop MapReduce. Hadoop MapReduce, while effective for batch processing, lacks the flexibility and speed of Spark, particularly when handling both batch and streaming data. Apache Flink, while a strong candidate for real-time data streaming, is more complex to set up and maintain in smaller-scale environments. Spark, on the other hand, provides a unified framework for both batch and stream processing, with a simple API and strong community support. Its ability to handle distributed data processing efficiently and integrate seamlessly with Kafka and MongoDB made Spark the optimal choice.

MongoDB

For data storage, MongoDB was selected over Cassandra and HBase. While Cassandra excels in write-heavy workloads and horizontal scalability, it uses a columnar data model, which is less flexible for the semi-structured, JSON-based data that our system generates. HBase, on the other hand, is well-suited for large-scale data storage but requires more complex configurations and is tightly coupled with the Hadoop ecosystem. MongoDB's document-oriented model, flexible schema, and ease of integration with Spark and Elasticsearch made it the most suitable choice for this project. Its ability to store, query, and scale document-based data efficiently was crucial for handling both raw and processed data in the system.[2]

3.3 Implementation steps

The implementation of this big data solution follows a series of key steps, beginning with real-time data ingestion, progressing through decoding and processing, and culminating in storage, indexing, and search functionality. Below, we detail each phase of the implementation.

Data Ingestion with Apache Kafka

The data pipeline begins with the real-time ingestion of data using Apache Kafka. Kafka acts as the primary message broker, collecting and transmitting data from various sources. The process is structured as follows:

Producer Configuration: Data producers are configured to send data in real-time to specific Kafka topics. Each Kafka message consists of multiple fields, such as filename, content, and timestamp. **Kafka Topics:** Kafka topics are defined based on the type and origin of data being ingested. These topics ensure that different data streams are isolated and processed independently. **Consumer Integration:** Spark Structured Streaming is integrated as the Kafka consumer, continuously reading data from the Kafka topics for further processing. This setup allows for high-throughput and fault-tolerant data ingestion, ensuring that data is reliably transmitted and available for downstream processing.

Data Decoding and Processing with PySpark

Once the data is ingested through Kafka, it is decoded and processed in real time using PySpark. Spark's distributed architecture allows the system to scale seamlessly, handling large volumes of data efficiently. The steps for decoding and processing are as follows:

Structured Streaming: PySpark's Structured Streaming API is used to consume data from Kafka. The incoming data is streamed as a continuous DataFrame, making it easy to apply transformations. **Custom Decoding Logic:** A custom decoding function is applied to the content field of each Kafka message. The content, initially encoded in a binary format, is decoded using specific parsing rules. This ensures that the raw data is transformed into a readable format. **Data Transformation:** After decoding, Spark applies necessary data transformations, such as filtering, aggregating, or enriching the data, to prepare it for storage. This real-time processing pipeline ensures that the data is decoded and ready for use as soon as it is ingested, allowing for rapid insights and decision-making.

Data Storage with MongoDB

After processing, the decoded data is stored in MongoDB, a document-oriented database. MongoDB's flexibility and scalability make it well-suited to handle both raw and processed data. The following steps outline the storage process:

MongoDB Collections: Data is stored in different MongoDB collections based on its type. For example, raw files and decoded files are stored in separate collections such as `rawfiles`, `decodedfiles`, or more specific collections like `sdpfiles`, `airfiles`, etc. **Storage in JSON Format:** The processed data, in JSON format, is written to MongoDB for easy retrieval and querying. Each record is indexed by fields such as `filename`, `timestamp`, and other metadata for efficient access. **Replication and Sharding:** MongoDB's replication and sharding features can be leveraged to ensure high availability and scalability, allowing the system to handle a growing volume of data as needed. This structured storage mechanism supports flexible queries and ensures that data is organized efficiently for retrieval and further processing.

Indexing with Elasticsearch

To enable fast and flexible search capabilities, the processed data stored in MongoDB is indexed in Elasticsearch. Elasticsearch allows for full-text search, aggregation, and querying capabilities, which are essential for real-time data analysis. The following steps outline the indexing process:

Integration with Spark: Spark is used to extract relevant data from MongoDB and push it to Elasticsearch. This integration ensures that every new record in MongoDB is indexed in Elasticsearch as part of the data pipeline. **Elasticsearch Indexing:** Data is indexed based on specific fields, such as **accountNumber**, `filename`, `content`, and `timestamp`. The indexing structure is designed to optimize search queries and support complex querying patterns. **Near Real-Time Search:** Elasticsearch provides near real-time indexing, allowing the system to reflect updates within milliseconds. This ensures that users can search for the latest data without delay. By indexing the data in Elasticsearch, the system provides advanced search capabilities, enabling users to perform detailed queries with minimal latency.

Web Client for Search with Flask

The final step in the implementation is the creation of a Flask-based web client, which allows users to search and retrieve documents stored in MongoDB and indexed in Elasticsearch. The web client acts as a user-friendly interface for interacting with the data. Key components of the web client include:

Flask Framework: Flask is chosen as the web framework due to its simplicity and flexibility, allowing for rapid development of RESTful APIs to interact with Elasticsearch and MongoDB. **Search API:** The Flask app provides a search API that allows

users to query Elasticsearch based on fields like accountNumber, filename, or content. The API interfaces directly with Elasticsearch to retrieve results and format them for display. Frontend Interface: The web client presents a simple search form where users can input search terms and filters. The search results, retrieved from Elasticsearch, are displayed in a user-friendly format. Pagination and Filtering: The Flask app supports features such as pagination, sorting, and filtering, making it easier for users to navigate large datasets and refine their search results. Real-Time Updates: With Elasticsearch's near real-time indexing, the web client is capable of displaying the most up-to-date information as soon as new data is indexed. This Flask web client provides a powerful, intuitive interface for users to interact with the data, making it easy to search and analyze large datasets.

3.4 Software architecture

The software architecture of this solution is designed as illustrated in the figure 3.1 to provide a scalable and efficient pipeline for real-time data ingestion, processing, storage, and retrieval. At the core of the architecture, Apache Kafka serves as the message broker, enabling high-throughput, low-latency data streaming from various sources. This ingested data is consumed and processed by PySpark, which handles real-time transformations and decoding of the raw data. The processed data is then stored in MongoDB, a NoSQL database that supports flexible schema designs and efficient querying. In parallel, the data is indexed in Elasticsearch, enabling advanced full-text search and filtering capabilities. The architecture is containerized using Docker, which ensures portability, easy deployment, and scalability across different environments. Lastly, a Flask-based web client interacts with Elasticsearch to provide a user-friendly interface for querying the data, allowing users like BI analysts and fraud detection agents to retrieve insights in near real-time. Each component is decoupled, allowing for independent scaling and maintenance, ensuring both flexibility and resilience in the system.

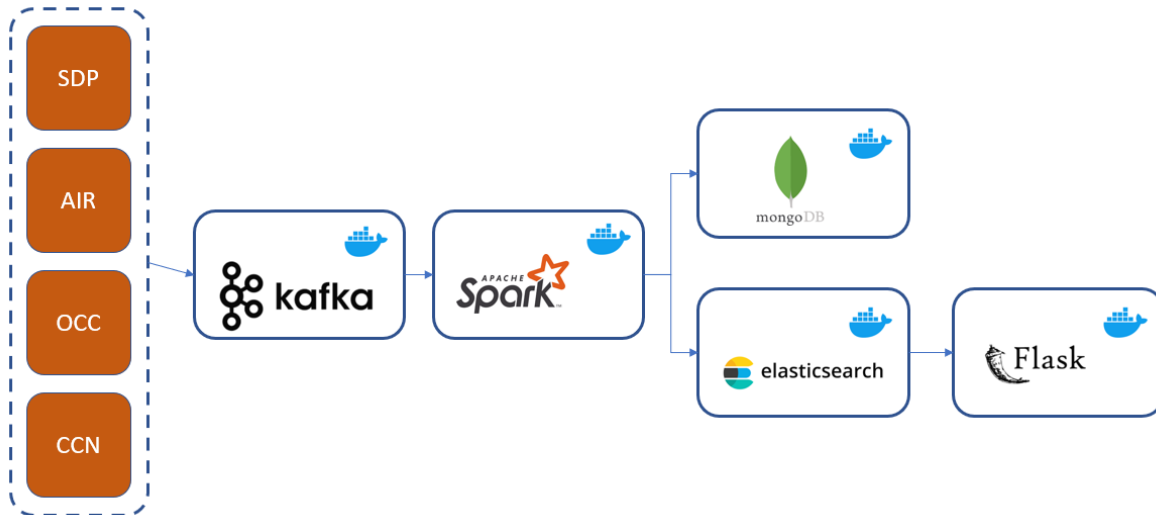


Figure 3.1: Software Architecture.

Conclusion

The implementation of this big data solution demonstrates the effective integration of modern technologies to build a scalable, real-time data processing pipeline. By utilizing Apache Kafka for high-throughput data ingestion, PySpark for distributed data decoding and transformation, MongoDB for flexible document storage, and Elasticsearch for powerful search indexing, the system is able to handle large volumes of data efficiently while ensuring quick and reliable access to stored information.

The addition of a Flask-based web client further enhances the user experience by providing an intuitive interface for querying and retrieving data. This architecture, built on containerized micro-services using Docker, not only meets the current performance and scalability requirements but also lays a strong foundation for future growth and extension. Each component was carefully selected after comparing alternative technologies, ensuring that the solution is optimized for performance, flexibility, and ease of maintenance.

In the following chapters, we will evaluate the system's performance, analyze potential challenges encountered during deployment, and discuss possible areas for improvement.

General Conclusion and Perspectives

This report has outlined the design, development, and implementation of a robust big data solution capable of handling real-time data ingestion, processing, storage, and retrieval. The system leverages cutting-edge technologies, such as Apache Kafka for data streaming, PySpark for distributed processing, MongoDB for scalable storage, Elasticsearch for advanced search indexing, and a Flask-based web client for user-friendly data access. Together, these components provide a flexible, scalable, and high-performance data pipeline that meets the demands of modern, data-driven organizations.

The solution has successfully demonstrated its ability to handle large volumes of data, ensuring low-latency processing and real-time availability of insights. Through the integration of Docker, the system is containerized and portable, offering the flexibility to deploy across various environments. Furthermore, each component was carefully selected after evaluating alternative technologies to ensure the solution is optimized for both current needs and future scalability.

As the landscape of big data and real-time processing continues to evolve, several opportunities arise to further enhance the solution and expand its applicability. Future iterations of the system could integrate machine learning models to provide more sophisticated insights. For instance, fraud detection agents could leverage machine learning algorithms trained on historical data to predict and identify potential fraudulent activities more accurately.

One of the most promising perspectives for this solution is its integration into the architecture of a charging system deployed within a Proxmox cluster. Proxmox, as a virtualization platform, offers an ideal environment to deploy this solution within a multi-node, clustered architecture. The scalability and high availability offered by Proxmox allow the big data pipeline to handle the increasing volumes of data that are

typical in a charging system environment.

The Docker containers housing Kafka, Spark, MongoDB, and Elasticsearch can be deployed across multiple virtual machines (VMs) within the Proxmox cluster. This deployment ensures load balancing and redundancy, which are critical for maintaining the performance and availability of the charging system.

Once the solution is integrated directly with the Core Charging System it can ingest data directly from the charging system, such as transactional data related to service usage, account balances, and billing events. With the real-time processing and storage capabilities of this big data pipeline, the charging system can benefit from instant data availability, enabling real-time analytics, fraud detection, and dynamic pricing models.

By integrating this big data solution into a Proxmox-powered charging system, organizations can enhance the real-time processing capabilities of their billing platforms, achieve higher availability, and improve resource utilization across the infrastructure. This integration also sets the foundation for future expansion, enabling seamless scaling and extending functionality to support advanced analytics, automation, and machine learning.

In conclusion, this solution represents a forward-thinking, scalable, and flexible approach to managing real-time data in a modern technological landscape. As the system evolves, its potential for integration into larger, more complex infrastructures such as Proxmox clusters makes it a valuable tool for organizations seeking to harness the full potential of their data assets.

Bibliography

- [1] Lucidcharts. Activity diagram, 2023. [Online; accessed 19-July-2024].
- [2] MongoDB. Mongoddb vs cassnadra, 2023. [Online; accessed 29-July-2024].
- [3] Wikipedia contributors. Asn.1 — Wikipedia, the free encyclopedia, 2024. [Online; accessed 19-July-2024].
- [4] Wikipédia. Apache kafka — wikipédia, l'encyclopédie libre, 2024. [En ligne; Page disponible le 4-avril-2024].